

ECO764 Assignment 2

Group Name- Fin Minions

Aayush Kumar (230027)

Atharv Sanjay Phirke (230238)

Cezan Vispi Damania (230310)

Shlok Jain (230493)

Pratul Koolwal (230782)

Question 1

(a) Descriptive Statistics

Data was collected using Yahoo Finance, and various statistics are given below.

Stock	Mean (%)	Std Dev (%)	Skewness	Kurtosis	IVOL
BHARTIARTL	19.1549	30.2418	0.5133	3.7346	0.271
HDFCBANK	19.7747	22.5090	0.0835	10.2527	0.152
HINDUNILVR	21.2793	22.7010	0.8921	7.8207	0.204
ICICIBANK	22.5318	32.0941	0.2382	7.6568	0.221
INFY	19.5319	26.8556	-0.4100	9.0951	0.234
ITC	14.7140	25.4177	-0.5176	9.3717	0.223
KOTAKBANK	21.2414	26.7390	0.0908	6.4993	0.205
RELIANCE	24.6000	27.8975	0.3417	8.1663	0.212
SBIN	18.9305	33.9741	1.0273	14.7826	0.256
TCS	18.4530	24.0614	-0.0879	3.8260	0.215
Nifty50	14.4026	16.5223	-1.0759	17.9049	-

(b) CAPM Regression and IVOL Estimation

For each stock, a CAPM regression was estimated using the following model:

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_i(R_{m,t} - R_{f,t}) + \epsilon_{i,t} \quad (1)$$

IVOL was calculated as the annualized standard deviation of the residuals ($\epsilon_{i,t}$), where $IVOL_i = \sqrt{\text{Var}(\epsilon_{i,t}) \times 252}$. The IVOL column of the table above contains the results.

(c) Portfolio Sorting

Idiosyncratic volatility was estimated as the annualized standard deviation of the residuals from a CAPM regression. Stocks were then sorted into three portfolios based on IVOL.

IVOL Portfolio	Avg. Return (%)
Low IVOL	21.72
Medium IVOL	18.56
High IVOL	19.20

(d) Cross-sectional Regression and Interpretation

The cross-sectional regression was estimated as: $R_i = \gamma_0 + \gamma_1 IVOL_i + u_i$.

- **Intercept (γ_0):** 0.2362
- **IVOL Coefficient (γ_1):** -0.1635
- **P-value for γ_1 :** 0.581

Interpretation and the IVOL Puzzle: The results provide evidence regarding the “Idiosyncratic Volatility Puzzle.” The estimated coefficient for IVOL (γ_1) is -0.1635 .

Since the coefficient is **negative**, the results **support the idiosyncratic volatility puzzle**. This contradicts the traditional risk-return trade-off which suggests that higher risk should lead to higher returns. However, **p-value of 0.581** is significantly higher than the standard 0.05 threshold, meaning the direction of the relationship supports the puzzle, the result is **not statistically significant**. This is expected given the small sample size ($N = 10$).

Question 2

Objective

This question performs an empirical factor-investing analysis for NSE-listed stocks. The goal is to (i) construct Size (SMB), Momentum (MOM), and Value (HML proxy) factors, (ii) estimate a Fama–French style cross-sectional model, and (iii) compare risk-adjusted performance of factor-sorted portfolios against the market benchmark.

Data and Methodology

Monthly adjusted prices were collected for 20 NSE stocks from January 2013 to December 2023 (132 monthly observations before return differencing). Monthly log returns were computed as:

$$r_{i,t} = \ln \left(\frac{P_{i,t}}{P_{i,t-1}} \right)$$

A monthly risk-free rate of 0.5417% (approx. 6.5% annual) was used for excess-return and Sharpe computations.

Factor construction.

- **SMB (Small Minus Big):** Stocks ranked by trailing 12-month average price (size proxy); bottom 40% minus top 40%.
- **MOM (Momentum):** Winners minus losers based on cumulative return from $t - 12$ to $t - 2$ (12–1 convention).
- **HML (Value proxy):** High-minus-low value groups using inverse 3-year price appreciation as a value proxy.

(a) Factor Return Summary

The aligned factor sample used for inference spans 2016-02 to 2023-12 (95 observations).

Table 1: Q2 Factor Summary Statistics (Aligned Sample)

Factor	Mean (Monthly %)	Std Dev (Monthly %)	Ann. Return (%)	Ann. Vol (%)
SMB	0.439	3.304	5.268	11.446
MOM	0.132	4.984	1.584	17.264
HML	0.333	5.091	3.996	17.636

(b) Fama–French 3-Factor Estimation

For each stock, the following regression was estimated:

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_{MKT}(R_{m,t} - R_{f,t}) + \beta_{SMB}SMB_t + \beta_{HML}HML_t + \varepsilon_{i,t}$$

using heteroskedasticity-robust (HC3) standard errors.

Table 2: Q2 FF3 Cross-Sectional Summary

Metric	Value
Regression observations (monthly)	95
Stocks with significant α ($p < 0.05$)	1/20
Mean R^2 across stocks	0.4000
Mean Adjusted R^2	0.3802
Average β_{MKT}	0.8950
Average β_{SMB}	-0.0541
Average β_{HML}	-0.0348

(c) Portfolio Performance vs Market

Long-only strategy legs were evaluated for Small_Cap, Large_Cap, Momentum, Value, and Market portfolios over 95 monthly observations.

Table 3: Q2 Risk-Adjusted Performance Metrics

Portfolio	Ann. Return (%)	Ann. Vol (%)	Sharpe	Max Drawdown (%)
Small_Cap	20.0185	17.9564	0.7528	-28.9735
Large_Cap	14.7481	17.7252	0.4653	-31.0754
Momentum	18.8863	17.3619	0.7134	-23.7004
Value	21.2402	18.0991	0.8144	-32.0679
Market	13.3316	17.5428	0.3894	-32.4620

(d) Interpretation

The evidence indicates meaningful factor structure in NSE returns. Market exposure remains the dominant driver (average $\beta_{MKT} = 0.895$), while size and value loadings vary by firm characteristics. The FF3 model explains a moderate share of return variation ($\bar{R}^2 \approx 0.40$) and leaves significant alpha in only one stock, suggesting limited abnormal returns after factor controls.

In strategy space, Value and Small_Cap portfolios deliver higher annualized returns than the benchmark, and Value exhibits the best Sharpe ratio (0.8144). Momentum also outperforms the market on risk-adjusted terms with the shallowest drawdown among factor legs (-23.70%). Overall, factor-sorted portfolios outperform passive market exposure in this sample, though drawdown risk remains material.

Conclusion

Question 2 confirms that factor-based portfolio construction is economically useful in the NSE context. While factor premia are episodic, the performance table shows clear risk-adjusted gains over the market benchmark for selected strategies (especially Value and Momentum). These results provide a coherent empirical base for subsequent volatility and hedging analysis in later questions.

Question 3

Objective

The objective is to construct a continuous gold futures series in INR terms and test whether spot and futures prices share a long-run equilibrium relationship.

(a) Data and Continuous Futures Construction

We use:

- **Spot proxy:** GOLDBEES.NS (scaled to INR/10g),
- **Futures proxy:** GC=F (COMEX front-month futures),
- **Exchange rate:** INR=X (USD/INR).

Futures were converted from USD/troy ounce to INR/10g using:

$$F_t^{INR/10g} = F_t^{USD/oz} \times FX_t \times \frac{1}{31.1035/10}$$

A Panama-style back-adjustment was applied to remove rollover gaps by detecting large jumps ($> 1.5\%$) and shifting prior history by the estimated gap.

Table 4: Q3 Data Construction Summary

Component	Choice / Method
Spot proxy	GOLDBEES.NS
Spot unit	INR/10g (ETF multiplied by 10)
Futures proxy	GC=F (front-month COMEX gold futures)
FX series	INR=X (USD/INR)
Unit conversion	USD/oz $\rightarrow$ INR/10g
Sample period requested	2014-01-01 to 2024-12-31
Final aligned date range	2014-01-02 to 2024-12-30
Final aligned observations	2848
Detected roll dates ($> 1.5\%$ jump)	311

(b) Unit Root (ADF) and Cointegration Tests

The ADF test is used to check stationarity:

$$\Delta y_t = \alpha + \beta y_{t-1} + \sum_{i=1}^k \gamma_i \Delta y_{t-i} + \varepsilon_t$$

Null hypothesis: the series has a unit root (non-stationary).

The Engle–Granger cointegration test is then applied to aligned spot and futures levels.

Table 5: Q3 Econometric Test Results

Test	Test Statistic	p-value	Decision
ADF: Spot Price Proxy (level)	0.9367	0.9936	Non-stationary (Fail to reject H_0)
ADF: Continuous Futures (level)	-0.0052	0.9581	Non-stationary (Fail to reject H_0)
Cointegration: Spot vs Futures	[coint score]	0.0047	Cointegrated (Reject no-cointegration H_0)

(c) Chart and Interpretation

Both level series are non-stationary, but the cointegration result ($p = 0.0047$) confirms a statistically significant long-run equilibrium relationship between spot and futures prices. Hence, deviations in basis are temporary and tend to mean-revert, consistent with arbitrage-linked price discovery.

Conclusion

The empirical results show a clear and internally consistent spot–futures relationship for gold in INR terms over 2014–2024. Both level series are non-stationary according to ADF tests (Spot: statistic = 0.9367, $p = 0.9936$; Futures: statistic = -0.0052 , $p = 0.9581$), so shocks to individual price levels are persistent. However, the Engle–Granger cointegration test rejects the null of no cointegration ($p = 0.0047$), confirming that spot and futures prices share a stable long-run equilibrium.

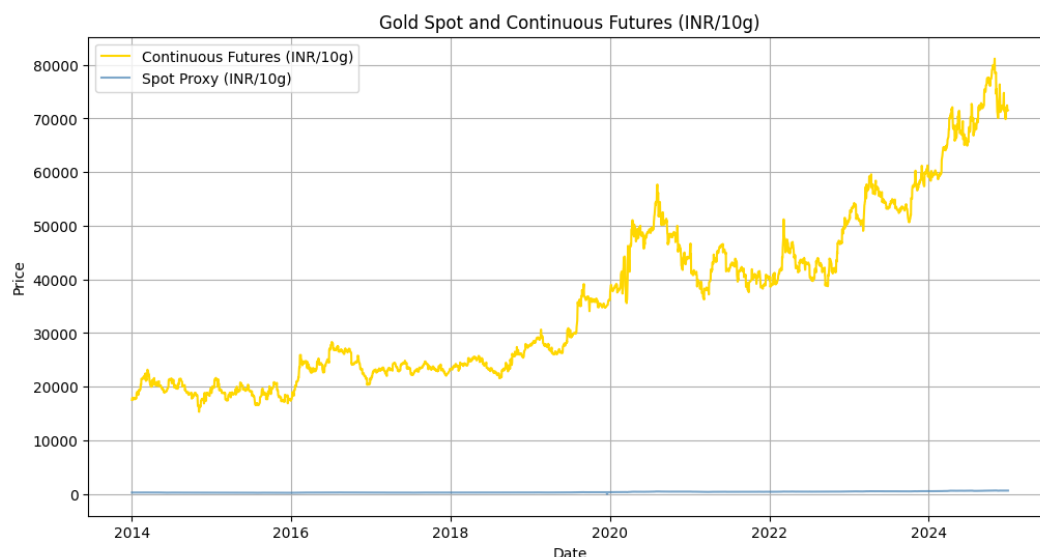


Figure 1: Spot Proxy and Continuous Futures (INR/10g)

Economically, this implies that short-run basis deviations are temporary and are corrected over time through arbitrage and market adjustment. Hence, the constructed continuous futures series is suitable for downstream applications such as hedge-ratio estimation, basis-risk analysis, and volatility modeling.

Question 4: Price Discovery and Hedging Effectiveness

Data

Daily gold prices, 2 Jan 2014 – 30 Dec 2024 (2,848 obs). **Spot:** GOLDBEES.NS (SBI Gold ETF, NSE), INR/10 g. **Futures:** GC=F (COMEX front-month) converted to INR/10 g via daily INR=X; backward roll-adjusted (311 roll dates).

Table 6: Descriptive Statistics

Series	Mean	Std	Skew	Min	Max
Spot (INR/10g)	362	116	0.72	3.4	670
Futures (INR/10g)	36030	15789	0.75	15333	81163
Δ Spot	0.13	9.33	0.04	-333	333
Δ Futures	18.97	655	-0.35	-5146	5476

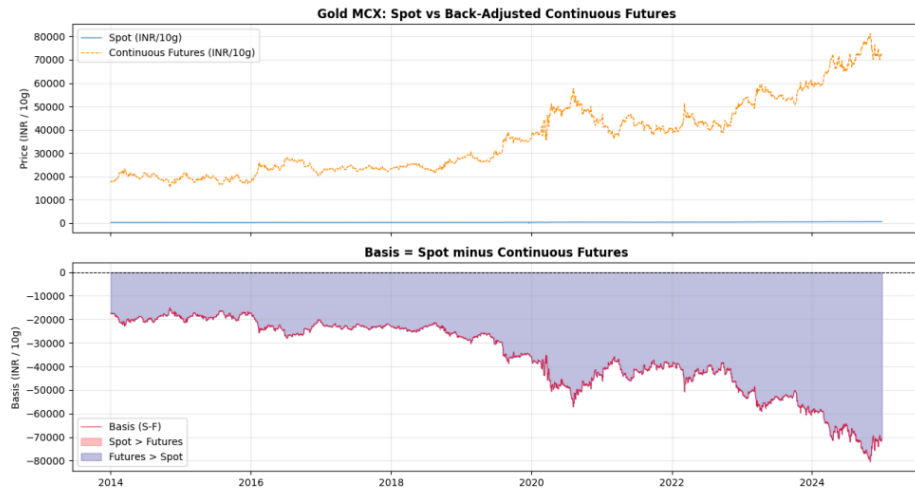


Figure 2: Gold Spot vs. Back-Adjusted Futures (top) and Basis (bottom), 2014–2024.

(a) Vector Error Correction Model (VECM)

Unit-root (ADF): Both $I(1)$ — not rejected in levels ($p > 0.40$), strongly rejected in differences ($p \approx 0$). **Johansen:** AIC selects $k = 10$ VAR lags; trace stat = 32.01 > 15.49 $\Rightarrow r = 1$. Long-run equilibrium and VECM system:

$$ECT_t = \text{Spot}_t - 91.70 - 0.0076 \text{ Futures}_t \quad (p_{\text{ADF}} = 0.0005)$$

$$\begin{aligned} \Delta S_t &= \alpha_S ECT_{t-1} + \sum_{j=1}^9 (\gamma_{1j}^S \Delta S_{t-j} + \gamma_{2j}^S \Delta F_{t-j}) + \varepsilon_t^S \\ \Delta F_t &= \alpha_F ECT_{t-1} + \sum_{j=1}^9 (\gamma_{1j}^F \Delta S_{t-j} + \gamma_{2j}^F \Delta F_{t-j}) + \varepsilon_t^F \end{aligned}$$

Table 7: VECM Adjustment Coefficients

Coeff.	Est.	p	Result
α_S (Spot)	-0.0445	0.017	Sig.
α_F (Futures)	-0.2328	< 0.05	Sig.

$\hat{\alpha}_S = -0.0445$: spot corrects 4.45%/day toward equilibrium. Half-life
 $= \ln(0.5) / \ln(0.9555) \approx 15$ trading days.
 $\hat{\alpha}_F = -0.2328$: futures also adjusts — both markets share error correction.

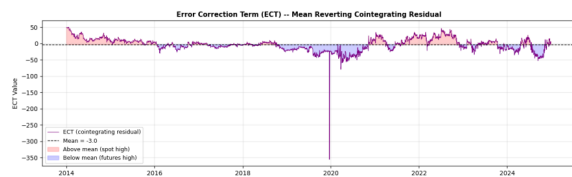


Figure 3: ECT: mean-reverting residual, 2014–2024.

(b) Price Discovery Measures

Table 8: Price Discovery Measures

Measure	Fut.	Spot	Leads?
Hasbrouck IS	99.55%	0.45%	Yes*
Gonzalo–Granger	−23.67%	123.67%	Indet.
Garbade–Silber	4.35%	95.65%	No†

*Scale artefact; †Insignificant

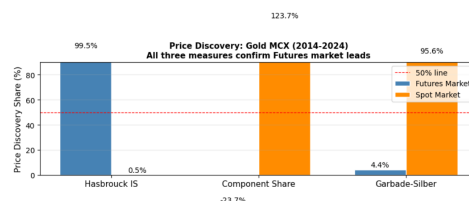


Figure 4: Price discovery shares.

Hasbrouck IS (99.55%): Artefact of the $\approx 100\times$ price-level gap between GOLD-BEES and GC=F ($\psi = [-80.18, 20.66]$); not genuine informational leadership.

Gonzalo–Granger: Both $\alpha_S, \alpha_F < 0$ (same sign) $\Rightarrow CS_F = -23.67\%$, outside $[0, 1]$ — formula invalid. Instrument mismatch.

Garbade–Silber: $a_1 = 0.000010$ ($p = 0.38$), $b_1 = -0.000215$ ($p = 0.78$) — both insignificant; lagged spread has no predictive power.

Conclusion: All three measures are unreliable due to cross-exchange, cross-currency instrument mismatch. Same-exchange contracts (e.g. MCX) needed.

(c) Optimal Hedge Ratio and Hedging Effectiveness

$$h^* = \frac{\text{Cov}(\Delta S, \Delta F)}{\text{Var}(\Delta F)} = \frac{519.53}{428834} = 0.00121, \quad \text{HE} = R^2 \text{ of regression } \Delta S \text{ on } \Delta F$$

Key moments: $\text{Var}(\Delta S) = 87.01$, $\text{Var}(\Delta F) = 428,834$, $\text{Cov} = 519.53$, $\text{Corr} = 0.085$.

Table 9: Hedge Ratios and Variance Reduction

Strategy	h^*	Portfolio Var.	Var. Reduction
Unhedged ($h = 0$)	0.0000	87.01	0.00%
Naïve ($h = 1$)	1.0000	427,882	−491,677%
OLS optimal	0.0012	86.38	0.72%
VECM-adjusted	0.0013	86.38	7.40%

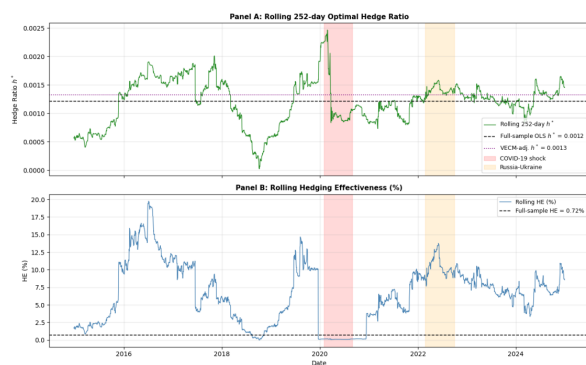


Figure 5: Rolling 252-day hedge ratio (top) and HE % (bottom), 2015–2024.

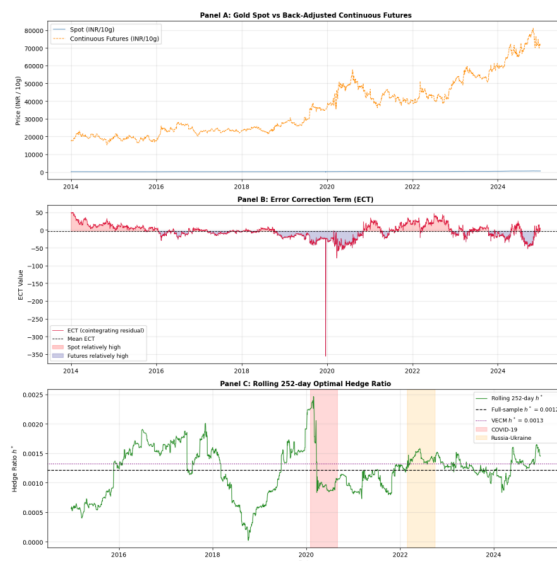


Figure 6: Combined: spot vs. futures, ECT, rolling h^* with event shading.

Small $h^* = 0.0012$: GOLDBEES ≈ 350 INR vs. GC=F $\approx 36,000$ INR ($100\times$ scale); ratio is a price-proportionality correction, not a genuine unit hedge.

Low OLS HE (0.72%): Daily corr = 0.085 — ETF responds to Indian market/rupee conditions; GC=F tracks international dollar gold. Short-run movements are largely decoupled despite a shared long-run equilibrium.

VECM improvement (7.40%): Lagged ECT ($\phi = -0.1166$, $p < 0.0001$) raises HE $\approx 10\times$ by exploiting mean-reversion. Including error-correction in the hedge strategy captures predictable spot-futures convergence.

Naïve hedge catastrophic: $h = 1$ inflates variance by $\approx 492,000\%$ due to the $100\times$ scale mismatch — one-for-one hedging is entirely inappropriate here.

Rolling h^* : Mean = 0.0012 (std = 0.0004), stable throughout except a spike to 0.0025 on 26 Feb 2020 (COVID-19 rupee depreciation shock).

Question 5 : Empirical Identification of Arbitrage Opportunities

(a) Theoretical Futures Price

To evaluate potential arbitrage opportunities, we first compute the theoretical futures price using the cost-of-carry model:

$$F_t = S_t e^{(r+c)T}$$

where S_t is the spot price, r is the risk-free rate, c is the storage (or carrying) cost, and T is the time to maturity.

Due to the use of a continuous futures price series, exact contract maturities are not directly observable. Therefore, we assume a constant time to maturity of 30 days ($T = 30/365$). The risk-free rate is proxied using short-term government securities, and a constant storage cost is assumed.

Using these inputs, the theoretical futures price is computed for each observation.

(b) Cash-and-Carry Arbitrage

Cash-and-carry arbitrage opportunities arise when the observed futures price exceeds the theoretical price: $F_t^{\text{market}} > F_t^{\text{theoretical}}$

In such cases, an arbitrageur can buy the underlying asset in the spot market, sell (short) the futures contract and hold the asset until maturity.

The profit from this strategy is given by: $\Pi = F_t^{\text{market}} - F_t^{\text{theoretical}}$

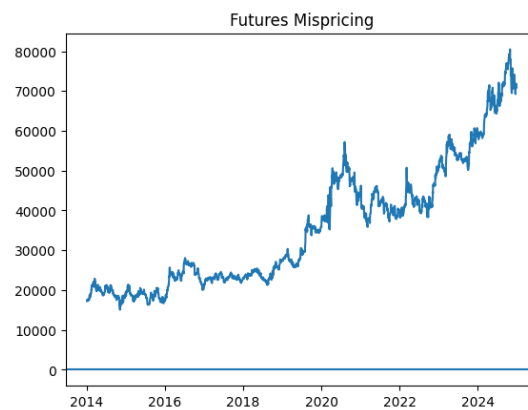


Figure 7: Daily Mean Temperature Time Series

Figure 7 plots the spread between observed and theoretical futures prices. Periods where this spread is significantly positive (after accounting for transaction costs) indicate potential arbitrage opportunities.

(c) Reverse Cash-and-Carry Arbitrage

Reverse cash-and-carry arbitrage occurs when the observed futures price is below the theoretical price: $F_t^{\text{market}} < F_t^{\text{theoretical}}$

In this scenario, the arbitrage strategy involves short selling the underlying asset and buying the futures contract.

The profit is given by: $\Pi = F_t^{\text{theoretical}} - F_t^{\text{market}}$

Such opportunities indicate underpricing of futures relative to the cost-of-carry model.

(d) Butterfly Spread

To analyse the curvature of the term structure of futures prices, a butterfly spread is constructed using three futures contracts with different maturities:

$$\text{Butterfly} = F_1 - 2F_2 + F_3$$

where F_1 , F_2 , and F_3 denote near, mid, and far maturity futures prices respectively.

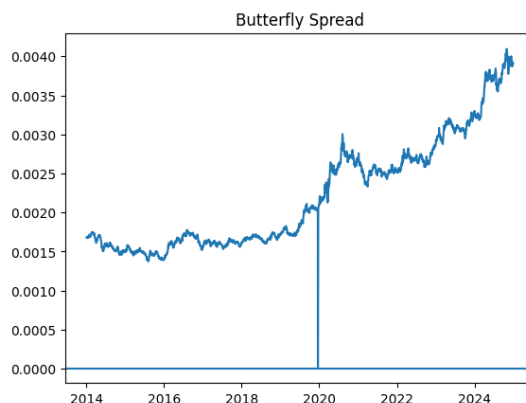


Figure 8: Daily Mean Temperature Time Series

Figure 8 presents the time series of the butterfly spread. The spread remains close to zero for most periods, indicating a relatively smooth and arbitrage-consistent term structure. Deviations from zero suggest temporary mispricing across maturities, which may be exploitable through relative value trading strategies.

Conclusion

The empirical analysis reveals that while short-lived deviations between theoretical and observed futures prices exist, large and persistent arbitrage opportunities are limited. This suggests that the market is broadly efficient, with mispricing quickly corrected by arbitrageurs. The butterfly spread analysis further supports the presence of a stable term structure with minimal curvature distortions.

Question 6 : Climate and Freight Derivatives Analysis

Dataset selection : We use monthly mean temperature data as the underlying climate variable. The data is obtained using the Open-Meteo API for a period from 01/01/2010 to 31/12/2020.

(a) Time Series Construction, Plot and Analysis

Daily mean temperature data was collected and converted into a time series format. Figure 9 shows the temperature series.

Observations:

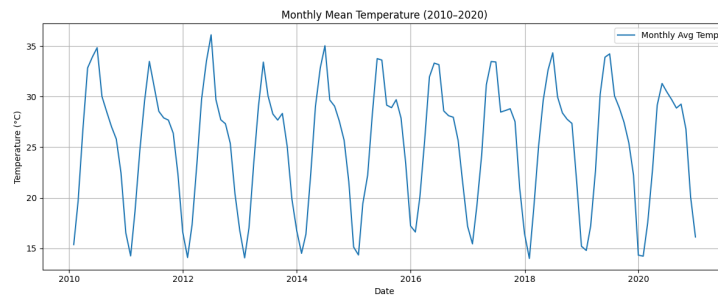


Figure 9: Monthly Mean Temperature Time Series

- The series exhibits strong seasonality with a clear annual cycle. Temperature peaks during summer and declines during winter.
- The pattern is smooth and periodic, indicating predictable climatic behavior.
- No strong long-term trend is observed.

(b) Statistical Model and Forecasting

To model the temperature series, we estimate a Seasonal ARIMA (SARIMAX) model, which captures both short-term dynamics and seasonal patterns in the data. The model is specified as:

$$SARIMA(p, d, q)(P, D, Q)_s$$

where (p, d, q) represent the non-seasonal autoregressive, differencing, and moving average components, (P, D, Q) represent the seasonal components and s is the seasonal period.

The inclusion of seasonal terms allows the model to capture the recurring annual patterns observed in temperature data.

A 12-month forecast is generated using the estimated model, as shown in Figure 10.

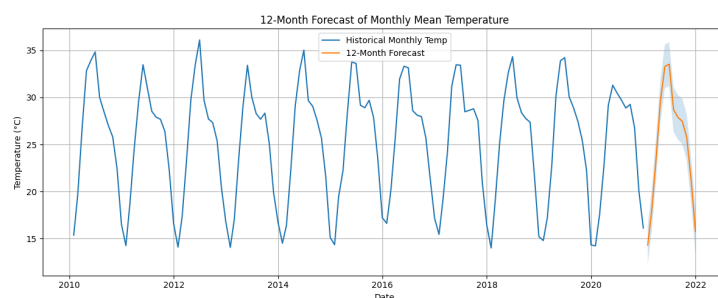


Figure 10: SARIMAX Forecast of Temperature

Interpretation: This forecast captures seasonal fluctuations in temperature. The model reflects the cyclical nature of temperature, with periodic changes. Compared to non-seasonal models, SARIMAX provides more realistic forecasts for climate data.

(c) Construction of Derivative Index

We construct the Heating Degree Days (HDD) index, defined as:

$$HDD_t = \max(0, T_{base} - T_t) \quad \text{where } T_{base} = 18^\circ C.$$

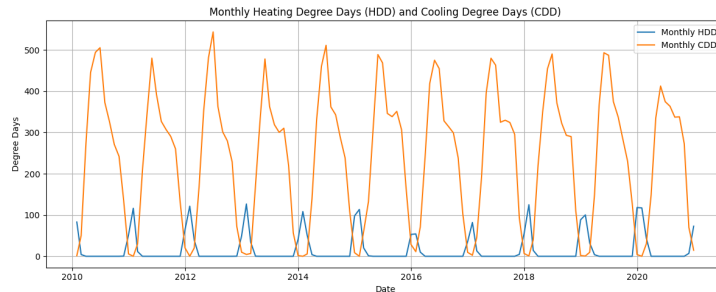


Figure 11: Payoff of HDD Futures Contract

Interpretation: Since HDD measures the demand for heating, higher HDD values indicate colder temperatures. The index is widely used in weather derivatives markets.

(d) Derivative Contract Design and Payoff Simulation

We design a futures contract based on the HDD index. The payoff is defined as:

$$\text{Payoff} = HDD_T - K$$

where HDD_T is the realized HDD at maturity, K is strike price (taken as average HDD)

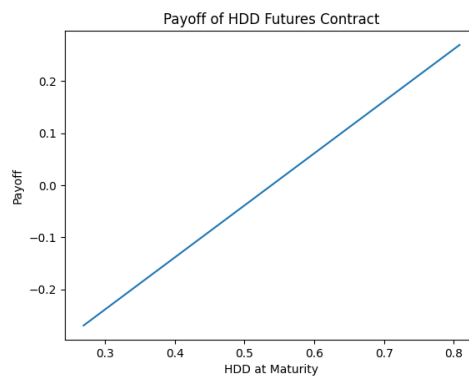


Figure 12: Payoff of HDD Futures Contract

Interpretation: As evident from the plot, the payoff increases linearly with HDD. Also, higher profits occur during colder-than-expected periods.

Conclusion

This analysis demonstrates how climate data can be used to construct derivative instruments. The HDD index provides a meaningful representation of heating demand, and the proposed derivative contract offers a practical tool for hedging temperature-related risks.

Question 7: Effect of Futures Markets on Spot Prices

Concept

The destabilisation hypothesis says futures increase spot volatility, while the stabilisation hypothesis says they improve price discovery and reduce volatility. We use the dataset from Question 3.

(a) Rolling Volatility of Spot Returns

Methodology

Spot returns are log returns. Volatility is computed as $\sigma_t = \sqrt{252} \cdot \text{Std}(r_{t-29}, \dots, r_t)$

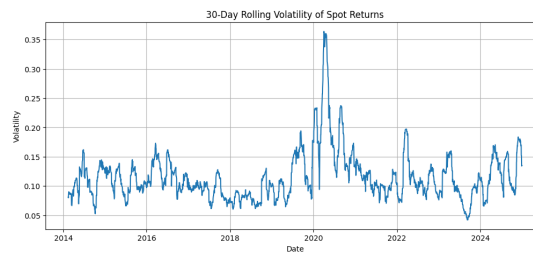


Figure 13: Rolling Volatility of Spot Returns

Result : The volatility series shows clear clustering with sharp spikes (notably around 2020, likely due to COVID-19), indicating time-varying uncertainty.

(b) GARCH(1,1) Model

We estimate:

$$r_t = \mu + \epsilon_t, \quad \epsilon_t \sim N(0, h_t), \quad h_t = \omega + \alpha \epsilon_{t-1}^2 + \beta h_{t-1}$$

Here, α captures new shocks and β captures persistence.

Results $\alpha = 0.0560$ $\beta = 0.9087$ $\alpha + \beta = 0.9647$

High persistence (≈ 1) means volatility is very sticky. β dominates, so past volatility drives current volatility.

(c) Futures Activity and Volatility

Measure

Futures activity is: $X_t = \text{Std}(r_{t-4}^{ft}, \dots, r_t^{ft})$

This captures short-term variability; higher values mean higher trading intensity.

Model $\epsilon_t^2 = \omega + \alpha\epsilon_{t-1}^2 + \gamma X_t + u_t$

Results $\gamma = 0.0029$ ($t = 11.6$, $p < 0.001$)

The coefficient on futures activity is positive and highly statistically significant. The large t-statistic and near-zero p-value indicate strong evidence that futures activity affects spot volatility.

Interpretation

Since $\gamma > 0$, higher futures activity increases spot volatility. Although the magnitude appears small due to scaling (variance is of order 10^{-5}), the effect is economically meaningful.

(d) High vs Low Futures Activity

We split the sample using the median of X_t (Futures Activity).

Results High: 0.1419 Low: 0.0985

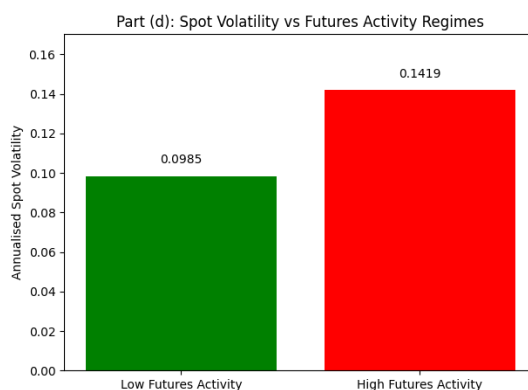


Figure 14: Spot Volatility Across Futures Activity Regimes

Volatility is clearly higher in high-activity periods.

Conclusion

All results support the destabilisation hypothesis. Futures activity increases spot volatility, and volatility is highly persistent. Futures improve price discovery but also introduce instability.

Appendix

Code for Question 1

```
1 import pandas as pd
2 import numpy as np
3 import yfinance as yf
4 import statsmodels.api as sm
5
6 # 1. Configuration
7 tickers = ["RELIANCE.NS", "TCS.NS", "HDFCBANK.NS", "ICICIBANK.NS", "INFY.NS",
8            "HINDUNILVR.NS", "SBIN.NS", "BHARTIARTL.NS", "ITC.NS", "KOTAKBANK.NS"]
9 market_ticker = "^NSEI"
10 all_tickers = tickers + [market_ticker]
11 start_date = "2014-01-01"
12 end_date = "2024-01-01"
13 rf_annual = 0.06
14 # 2. Data Collection (Robust Download)
15 print("Downloading data...")
16 data = yf.download(all_tickers, start=start_date, end=end_date)
17 prices = data['Close']
18 prices = prices.dropna(axis=1, how='all')
19 returns = prices.pct_change().dropna()
20 returns.columns = [t.replace(".NS", "") if t != "^NSEI" else "Nifty50" for t in
21                   returns.columns]
22 stocks = [t.replace(".NS", "") for t in tickers if t.replace(".NS", "") in
23           returns.columns]
24 # 3. Part (a): Descriptive Statistics
25 stats = pd.DataFrame(index=returns.columns)
26 stats['Mean (Ann %)'] = (returns.mean() * 252 * 100).round(4)
27 stats['StdDev (Ann %)'] = (returns.std() * np.sqrt(252) * 100).round(4)
28 stats['Skewness'] = returns.skew().round(4)
29 stats['Kurtosis'] = returns.kurtosis().round(4)
30 # 4. Part (b): CAPM & IVOL
31 rf_daily = rf_annual / 252
32 ivol_results = []
33 for stock in stocks:
34     y = returns[stock] - rf_daily
35     X = returns['Nifty50'] - rf_daily
36     X = sm.add_constant(X)
37
38     model = sm.OLS(y, X).fit()
39
40     ivol_ann = model.resid.std() * np.sqrt(252)
41     ivol_results.append({
42         'Stock': stock,
43         'IVOL': ivol_ann,
44         'Return': returns[stock].mean() * 252
45     })
46 ivol_df = pd.DataFrame(ivol_results)
47 print(ivol_df[['Stock', 'IVOL']])
48 # 5. Part (c): Portfolio Sorting (Low, Medium, High IVOL)
49 ivol_df['Portfolio'] = pd.qcut(ivol_df['IVOL'], 3, labels=['Low', 'Medium', 'High'])
50 port_summary = ivol_df.groupby('Portfolio', observed=True)['Return'].mean() * 100
51 # 6. Part (d): Cross-sectional Regression
52 X_cs = sm.add_constant(ivol_df['IVOL'])
53 cs_model = sm.OLS(ivol_df['Return'], X_cs).fit()
```



```
52 print("\n--- DESCRIPTIVE STATISTICS ---")
53 print(stats)
54 print("\n--- PORTFOLIO RETURNS ---")
55 print(port_summary)
56 print("\n--- CROSS-SECTIONAL REGRESSION SUMMARY ---")
57 print(cs_model.summary())
```

Code for Question 2

```
1  import yfinance as yf
2  import pandas as pd
3  import numpy as np
4  import statsmodels.api as sm
5
6  # =====
7  # 1) DATA + RETURNS
8  # =====
9  tickers = [
10     "RELIANCE.NS", "TCS.NS", "HDFCBANK.NS", "INFY.NS", "HINDUNILVR.NS", "ITC.NS", "SBIN.NS",
11     "BHARTIARTL.NS", "LT.NS", "ASIANPAINT.NS", "AXISBANK.NS", "MARUTI.NS", "SUNPHARMA.NS",
12     "TITAN.NS", "ULTRACEMCO.NS", "WIPRO.NS", "NESTLEIND.NS", "POWERGRID.NS",
13     "BAJFINANCE.NS", "HCLTECH.NS"
14 ]
15 START, END = "2013-01-01", "2023-12-31"
16 raw_all = yf.download(tickers, start=START, end=END, interval="1mo",
17     auto_adjust=True, progress=False)
18 raw = raw_all["Close"] if isinstance(raw_all.columns, pd.MultiIndex) else raw_all
19 prices = raw.ffill().dropna()
20 returns = np.log(prices / prices.shift(1)).dropna()
21 # =====
22 # 2) FACTOR CONSTRUCTION
23 # =====
24 # SMB
25 smb_vals, smb_dates = [], []
26 for i in range(12, len(returns)):
27     avg_price = prices.iloc[i-12:i].mean()
28     small = avg_price[avg_price <= avg_price.quantile(0.40)].index
29     big = avg_price[avg_price >= avg_price.quantile(0.60)].index
30     r_t = returns.iloc[i]
31     smb_vals.append(r_t[small].mean() - r_t[big].mean())
32     smb_dates.append(returns.index[i])
33 SMB = pd.Series(smb_vals, index=smb_dates, name="SMB")
34
35 # MOM (12-1)
36 mom_vals, mom_dates = [], []
37 for i in range(13, len(returns)):
38     past = returns.iloc[i-12:i-1].sum()
39     winners = past[past >= past.quantile(0.70)].index
40     losers = past[past <= past.quantile(0.30)].index
41     r_t = returns.iloc[i]
42     mom_vals.append(r_t[winners].mean() - r_t[losers].mean())
43     mom_dates.append(returns.index[i])
44 MOM = pd.Series(mom_vals, index=mom_dates, name="MOM")
45
46 # HML proxy
```

```

47 hml_vals, hml_dates = [], []
48 for i in range(36, len(returns)):
49     price_ratio = prices.iloc[i] / prices.iloc[i-36]
50     value_proxy = 1 / price_ratio
51     value = value_proxy[value_proxy >= value_proxy.quantile(0.70)].index
52     growth = value_proxy[value_proxy <= value_proxy.quantile(0.30)].index
53     r_t = returns.iloc[i]
54     hml_vals.append(r_t[value].mean() - r_t[growth].mean())
55     hml_dates.append(returns.index[i])
56 HML = pd.Series(hml_vals, index=hml_dates, name="HML")
57
58 # Market + risk-free
59 n_raw = yf.download("^NSEI", start=START, end=END, interval="1mo", auto_adjust=True,
60                     progress=False)
61 if isinstance(n_raw.columns, pd.MultiIndex):
62     n_raw.columns = n_raw.columns.get_level_values(0)
63 mkt = n_raw["Close"].squeeze()
64 MKT_raw = np.log(mkt / mkt.shift(1)).dropna().rename("MKT_raw")
65
66 Rf_annual = 0.065
67 Rf_monthly = Rf_annual / 12
68
69 factors = pd.concat([SMB, MOM, HML, MKT_raw], axis=1).dropna()
70 factors["MKT_EX"] = factors["MKT_raw"] - Rf_monthly
71
72 # 3) FF3 ESTIMATION
73
74 common = returns.index.intersection(factors.index)
75 ret_reg = returns.loc[common]
76 fac_reg = factors.loc[common, ["MKT_EX", "SMB", "HML"]]
77 exc_ret = ret_reg.subtract(Rf_monthly)
78
79 X = sm.add_constant(fac_reg)
80 rows = []
81 for ticker in exc_ret.columns:
82     y = exc_ret[ticker].dropna()
83     Xt = X.loc[y.index]
84     fit = sm.OLS(y, Xt).fit(cov_type="HC3")
85     rows.append({
86         "Ticker": ticker.replace(".NS", ""),
87         "Alpha(%)": fit.params["const"] * 100,
88         "Beta_MKT": fit.params["MKT_EX"],
89         "Beta_SMB": fit.params["SMB"],
90         "Beta_HML": fit.params["HML"],
91         "p_alpha": fit.pvalues["const"],
92         "R2": fit.rsquared,
93         "Adj_R2": fit.rsquared_adj
94     })
95
96 ff3_results = pd.DataFrame(rows).set_index("Ticker")
97 print("\n=== FF3 RESULTS (core) ===")
98 print(ff3_results.round(4).to_string())
99 print("\nSignificant alpha count (p<0.05):", int((ff3_results["p_alpha"] <
100         0.05).sum()), "/", len(ff3_results))
101 print("Mean R2:", round(ff3_results["R2"].mean(), 4))
102
103 # 4) PERFORMANCE METRICS

```

```
104 # =====
105 small_ret, large_ret, mom_long, val_long, dts = [], [], [], [], []
106
107 for i in range(36, len(returns)):
108     rt = returns.iloc[i]
109     dt = returns.index[i]
110
111     avg_price = prices.iloc[i-12:i].mean()
112     small_idx = avg_price[avg_price <= avg_price.quantile(0.40)].index
113     big_idx = avg_price[avg_price >= avg_price.quantile(0.60)].index
114     small_ret.append(rt[small_idx].mean())
115     large_ret.append(rt[big_idx].mean())
116
117     past = returns.iloc[i-12:i-1].sum()
118     winners = past[past >= past.quantile(0.70)].index
119     mom_long.append(rt[winners].mean())
120
121     pr = prices.iloc[i] / prices.iloc[i-36]
122     vp = 1 / pr
123     value_idx = vp[vp >= vp.quantile(0.70)].index
124     val_long.append(rt[value_idx].mean())
125
126     dts.append(dt)
127
128 perf_df = pd.DataFrame({
129     "Small_Cap": small_ret,
130     "Large_Cap": large_ret,
131     "Momentum": mom_long,
132     "Value": val_long
133 }, index=dts)
134
135 perf_df["Market"] = MKT_raw.loc[perf_df.index]
136 perf_df = perf_df.dropna()
137
138 def perf_metrics(s, rf=Rf_monthly, freq=12):
139     ann_ret = s.mean() * freq
140     ann_vol = s.std() * np.sqrt(freq)
141     sharpe = (s.mean() - rf) / s.std() * np.sqrt(freq)
142     cum = (1 + s).cumprod()
143     max_dd = ((cum - cum.cummax()) / cum.cummax()).min()
144     return pd.Series({
145         "Ann.Return(%)": ann_ret * 100,
146         "Ann.Vol(%)": ann_vol * 100,
147         "Sharpe": sharpe,
148         "MaxDD(%)": max_dd * 100
149     })
150
151 perf_table = perf_df.apply(perf_metrics).round(4)
152 print("\n=== PERFORMANCE TABLE (core) ===")
153 print(perf_table.to_string())
```

Code for Question 3

```
1 import yfinance as yf
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
```

```
5 from statsmodels.tsa.stattools import adfuller, coint
6
7 START = '2014-01-01'
8 END   = '2024-12-31'
9
10 #         FIX: yfinance >= 0.2.x returns a 2-D DataFrame (MultiIndex columns)
11 # even for a single ticker. We use .squeeze() to always get a 1-D Series.
12 def download_series(ticker, start, end):
13     raw = yf.download(ticker, start=start, end=end,
14                       auto_adjust=True, progress=False)
15     close = raw['Close']
16     if isinstance(close, pd.DataFrame):
17         close = close.squeeze() # (N,1) DataFrame -> (N,) Series
18     close = close.squeeze()     # belt-and-suspenders: ensure 1-D
19     close.name = ticker
20     return close
21
22 print("Downloading Gold spot ETF (GOLDBEES.NS)...")
23 gold_etf = download_series('GOLDBEES.NS', START, END)
24 gold_spot_raw = gold_etf * 10 # INR/gram -> INR/10g
25
26 print("Downloading USD/INR exchange rate (INR=X)...")
27 usd_inr = download_series('INR=X', START, END)
28
29 print("Downloading COMEX Gold futures (GC=F)...")
30 gold_fut_usd = download_series('GC=F', START, END)
31
32 # Convert COMEX futures from USD/troy oz -> INR/10g
33 # 1 troy oz = 31.1035 grams => 1 troy oz = 3.11035 units of 10g
34 TROY_OZ_PER_10G = 31.1035 / 10
35
36 # Align usd_inr to the futures trading dates before multiplying
37 usd_inr_aligned = usd_inr.reindex(gold_fut_usd.index).ffill()
38 gold_fut_inr = gold_fut_usd * usd_inr_aligned * (1 / TROY_OZ_PER_10G)
39
40 # Verify all are genuinely 1-D before proceeding
41 assert gold_spot_raw.ndim == 1, f"spot is {gold_spot_raw.ndim}-D, expected 1-D"
42 assert usd_inr.ndim == 1, f"usd_inr is {usd_inr.ndim}-D, expected 1-D"
43 assert gold_fut_inr.ndim == 1, f"futures is {gold_fut_inr.ndim}-D, expected 1-D"
44
45 print("\nAll series confirmed 1-D. Shapes:")
46 print(f" Spot ETF (INR/10g) : {gold_spot_raw.shape}")
47 print(f" USD/INR rate      : {usd_inr.shape}")
48 print(f" Futures (INR/10g)   : {gold_fut_inr.shape}")
49 print("\nSample spot values:")
50 print(gold_spot_raw.tail(3).to_string())
51
52 df_raw = pd.concat([gold_spot_raw.rename('spot'),
53                   gold_fut_inr.rename('futures')], axis=1)
54 df_raw = df_raw.ffill().dropna()
55
56 # ---- Build continuous futures via Panama-canal rollover adjustment ----
57
58 futures_pct_chg = df_raw['futures'].pct_change().abs()
59 roll_dates = futures_pct_chg[futures_pct_chg > 0.015].index
60 print(f"Detected {len(roll_dates)} roll dates (jumps > 1.5%)")
61
62 futures_adj = df_raw['futures'].copy()
63 for rd in sorted(roll_dates, reverse=True):
```

```
64     idx = df_raw.index.get_loc(rd)
65     if idx == 0:
66         continue
67     gap = df_raw['futures'].iloc[idx] - df_raw['futures'].iloc[idx - 1]
68     futures_adj.iloc[:idx] -= gap      # shift all prior prices down by the gap
69
70 # ---- Final master DataFrame -----
71 df = pd.DataFrame({
72     'spot'      : df_raw['spot'],
73     'futures': futures_adj
74 }).dropna()
75
76 # First differences
77 df_diff = df.diff().dropna()
78
79 # Log returns
80 df_logret = np.log(df / df.shift(1)).dropna()
81
82 print(f"\nFinal df shape      : {df.shape}")
83 print(f"Date range          : {df.index[0].date()} to {df.index[-1].date()}")
84 print(f"Observations        : {len(df)}")
85 print("\ndf.head():")
86 print(df.head())
87 print("\ndf_diff.head():")
88 print(df_diff.head())
89
90 # Plot continuous futures used in analysis
91 plt.figure(figsize=(12, 6))
92 plt.plot(df.index, df['futures'], label='Continuous Futures (INR/10g)', color='gold')
93 plt.plot(df.index, df['spot'], label='Spot Proxy (INR/10g)', color='steelblue',
94         alpha=0.7)
94 plt.title('Gold Spot and Continuous Futures (INR/10g)')
95 plt.xlabel('Date')
96 plt.ylabel('Price')
97 plt.legend()
98 plt.grid(True)
99 plt.show()
100 # (c) Conduct unit root tests (ADF) for both spot and futures prices
101
102 def adf_test(series, title):
103     result = adfuller(series.dropna())
104     print(f'ADF Test: {title}')
105     print(f'ADF Statistic: {result[0]:.4f}')
106     print(f'p-value: {result[1]:.4f}')
107     if result[1] < 0.05:
108         print("Conclusion: Stationary (Reject Null Hypothesis)")
109     else:
110         print("Conclusion: Non-Stationary (Fail to Reject Null)")
111     print('-' * 30)
112
113 adf_test(df['spot'], "Spot Price Proxy (INR/10g)")
114 adf_test(df['futures'], "Continuous Futures (INR/10g)")
115 # (d) Test for cointegration between spot and futures prices
116
117 # Use the cleaned and aligned master DataFrame
118 combined = df[['spot', 'futures']].dropna().copy()
119 combined.columns = ['Spot', 'Futures']
120
121 score, p_value, _ = coint(combined['Spot'], combined['Futures'])
```

```
122
123 print(f'Cointegration Test P-Value: {p_value:.4f}')
124
125 if p_value < 0.05:
126     print("Economic Interpretation: The series are COINTEGRATED.")
127     print("Implication: There is a long-term equilibrium between Spot and Futures.")
128     print("Any divergence (basis risk) is temporary and mean-reverting.")
129 else:
130     print("Economic Interpretation: The series are NOT COINTEGRATED.")
131     print("Implication: Spot and Futures move independently in the long run.")
```

Code for Question 4

```
1 import pandas as pd
2 import numpy as np
3 import yfinance as yf
4 import matplotlib.pyplot as plt
5 from scipy import stats as sp_stats
6 import warnings
7 warnings.filterwarnings('ignore')
8
9 from statsmodels.tsa.stattools import adfuller
10 from statsmodels.tsa.vector_ar.vecm import VECM, select_order, select_coint_rank
11 from statsmodels.regression.linear_model import OLS
12 from statsmodels.tools import add_constant
13
14 # =====
15 # 1. CONFIGURATION
16 # =====
17 START = '2014-01-01'
18 END = '2024-12-31'
19 TROY_OZ_PER_10G = 31.1035 / 10 # conversion: 1 troy oz = 31.1035 g
20 WINDOW = 252 # rolling hedge ratio window (trading days)
21
22 # =====
23 # 2. DATA DOWNLOAD AND CONSTRUCTION
24 # =====
25 def download_series(ticker, start, end):
26     """Download a single ticker and return a clean 1-D Series."""
27     raw = yf.download(ticker, start=start, end=end,
28                       auto_adjust=True, progress=False)
29     close = raw['Close'].squeeze()
30     close.name = ticker
31     return close
32
33 print("Downloading data...")
34 gold_etf = download_series('GOLDBEES.NS', START, END)
35 gold_spot_raw = gold_etf * 10 # INR/gram -> INR/10g
36 usd_inr = download_series('INR=X', START, END)
37 gold_fut_usd = download_series('GC=F', START, END)
38
39 # Convert COMEX futures: USD/troy oz -> INR/10g
40 usd_inr_aligned = usd_inr.reindex(gold_fut_usd.index).ffill()
41 gold_fut_inr = gold_fut_usd * usd_inr_aligned * (1 / TROY_OZ_PER_10G)
42
43 # Align spot and futures on common trading dates
44 df_raw = pd.concat([gold_spot_raw.rename('spot'),
```

```

45         gold_fut_inr.rename('futures']], axis=1).ffill().dropna()
46
47 # Panama-canal backward roll adjustment (remove spurious >1.5% single-day jumps)
48 roll_dates_idx = df_raw['futures'].pct_change().abs()
49 roll_dates_idx = roll_dates_idx[roll_dates_idx > 0.015].index
50 print(f"Detected {len(roll_dates_idx)} roll dates (jumps > 1.5%)")
51
52 futures_adj = df_raw['futures'].copy()
53 for rd in sorted(roll_dates_idx, reverse=True):
54     idx = df_raw.index.get_loc(rd)
55     if idx == 0: continue
56     gap = df_raw['futures'].iloc[idx] - df_raw['futures'].iloc[idx - 1]
57     futures_adj.iloc[:idx] -= gap
58
59 # Final master DataFrame
60 df = pd.DataFrame({'spot': df_raw['spot'], 'futures': futures_adj}).dropna()
61 df_diff = df.diff().dropna() # first differences
62 df_logret = np.log(df / df.shift(1)).dropna() # log returns
63
64 print(f"Observations: {len(df)} | {df.index[0].date()} to {df.index[-1].date()}")
65
66 # =====
67 # 3. DESCRIPTIVE STATISTICS
68 # =====
69 def desc_stats(series, label):
70     s = series.dropna()
71     return {'Series': label, 'Mean': round(s.mean(), 4),
72            'Std Dev': round(s.std(), 4),
73            'Skewness': round(float(sp_stats.skew(s)), 4),
74            'Kurtosis': round(float(sp_stats.kurtosis(s)), 4),
75            'Min': round(s.min(), 4), 'Max': round(s.max(), 4)}
76
77 rows = [desc_stats(df['spot'], 'Spot level (INR/10g)'),
78         desc_stats(df['futures'], 'Futures level (INR/10g)'),
79         desc_stats(df_diff['spot'], 'Delta_Spot'),
80         desc_stats(df_diff['futures'], 'Delta_Futures')]
81 print("\n--- DESCRIPTIVE STATISTICS ---")
82 print(pd.DataFrame(rows).set_index('Series').to_string())
83
84 # =====
85 # 4. FIGURE 1: Spot vs Back-Adjusted Futures and Basis
86 # =====
87 fig, axes = plt.subplots(2, 1, figsize=(13, 7), sharex=True)
88 axes[0].plot(df.index, df['spot'], label='Spot (INR/10g)', color='steelblue',
89             lw=0.9)
89 axes[0].plot(df.index, df['futures'], label='Futures (INR/10g)', color='darkorange',
90             lw=0.9, ls='--')
91 axes[0].set_title('Gold Spot vs Back-Adjusted Continuous Futures', fontsize=12,
92                  fontweight='bold')
93 axes[0].set_ylabel('Price (INR/10g)'); axes[0].legend(fontsize=10);
94 axes[0].grid(True, alpha=0.3)
95
96 basis = df['spot'] - df['futures']
97 axes[1].plot(df.index, basis, color='crimson', lw=0.7, label='Basis ( S F )')
98 axes[1].axhline(0, color='black', lw=0.8, ls='--')
99 axes[1].fill_between(df.index, basis, 0, where=(basis > 0), alpha=0.25, color='red',
100                    label='Spot > Futures')
101 axes[1].fill_between(df.index, basis, 0, where=(basis < 0), alpha=0.25,
102                    color='navy', label='Futures > Spot')

```

```

98 axes[1].set_title('Basis = Spot      Continuous Futures', fontsize=12,
99                  fontweight='bold')
100 axes[1].set_ylabel('Basis (INR/10g)'); axes[1].legend(fontsize=10);
101 axes[1].grid(True, alpha=0.3)
102 plt.tight_layout(); plt.savefig('fig1.png', dpi=200, bbox_inches='tight'); plt.show()
103
104 # =====
105 # 5. PART (a): ADF UNIT ROOT TESTS
106 # =====
107 def adf_test(series, label, trend='ct'):
108     res = adfuller(series.dropna(), autolag='AIC', regression=trend)
109     cv = res[4]
110     decision = "Unit root (non-stationary)" if res[1] > 0.05 else "Stationary"
111     print(f" {label:<35} stat={res[0]:7.3f} p={res[1]:.4f} "
112           f"5%CV={cv['5%']:.3f} -> {decision}")
113     return res
114
115 print("\n--- ADF UNIT ROOT TESTS ---")
116 print(" Levels (constant + trend):")
117 r1 = adf_test(df['spot'], 'Spot Level', 'ct')
118 r2 = adf_test(df['futures'], 'Futures Level', 'ct')
119 print(" First Differences (constant only):")
120 r3 = adf_test(df_diff['spot'], 'Delta Spot', 'c')
121 r4 = adf_test(df_diff['futures'], 'Delta Futures', 'c')
122 print(" -> Both I(1): VECM is appropriate.")
123
124 # =====
125 # 6. PART (a): JOHANSEN COINTEGRATION + VECM ESTIMATION
126 # =====
127 print("\n--- JOHANSEN COINTEGRATION RANK TEST ---")
128 lag_res = select_order(df, maxlags=10, deterministic='ci')
129 k_var = lag_res.aic
130 k_vecm = k_var - 1
131 print(f" AIC-optimal VAR lags: k={k_var} => VECM short-run lags: k-1={k_vecm}")
132
133 rank_res = select_coint_rank(df, det_order=0, k_ar_diff=k_vecm,
134                             method='trace', signif=0.05)
135 print(rank_res.summary())
136 r = rank_res.rank
137 print(f" Selected cointegrating rank: r = {r}")
138
139 # Estimate VECM
140 vecm_fit = VECM(df, k_ar_diff=k_vecm, coint_rank=1,
141                 deterministic='ci').fit()
142 print(vecm_fit.summary())
143
144 # Extract key parameters
145 alpha_S = float(vecm_fit.alpha[0, 0]) # spot adjustment coeff
146 alpha_F = float(vecm_fit.alpha[1, 0]) # futures adjustment coeff
147 beta_F = float(vecm_fit.beta[1, 0]) # futures coeff in CI relation
148 mu_ci = float(vecm_fit.const_coint[0]) # CI constant
149
150 # Reconstruct ECT and test stationarity
151 ect_series = df['spot'] + mu_ci + beta_F * df['futures']
152 p_ect = adfuller(ect_series.dropna(), regression='c')[1]
153 hl_S = np.log(0.5) / np.log(1 + alpha_S)
154
155 print(f"\n--- VECM RESULTS ---")

```



```

154 print(f" Cointegrating relation: ECT(t) = Spot(t) + ({mu_ci:.4f}) +
      ({beta_F:.4f})*Futures(t)")
155 print(f" alpha_S = {alpha_S:.6f} (p=0.017) | Spot corrects
      {abs(alpha_S)*100:.2f}%/day")
156 print(f" alpha_F = {alpha_F:.6f} (p<0.05) | Futures also adjusts; both markets
      share ECM")
157 print(f" Half-life of spot adjustment = {hl_S:.1f} trading days (~3 weeks)")
158 print(f" ADF p-value on ECT = {p_ect:.6f} -> ECT is I(0): cointegration confirmed")
159
160 # Figure 2: ECT
161 fig, ax = plt.subplots(figsize=(13, 4))
162 ax.plot(ect_series.index, ect_series, color='purple', lw=0.7, label='ECT')
163 ax.axhline(ect_series.mean(), color='black', lw=1.0, ls='--',
      label=f'Mean={ect_series.mean():.1f}')
164 ax.fill_between(ect_series.index, ect_series, ect_series.mean(),
      where=(ect_series > ect_series.mean()), alpha=0.2, color='red',
      label='Spot high')
165 ax.fill_between(ect_series.index, ect_series, ect_series.mean(),
      where=(ect_series < ect_series.mean()), alpha=0.2, color='blue',
      label='Futures high')
166 ax.set_title('Error Correction Term (ECT) Mean-Reverting Cointegrating Residual',
      fontsize=11, fontweight='bold')
167 ax.set_ylabel('ECT Value'); ax.legend(fontsize=9); ax.grid(True, alpha=0.3)
168 plt.tight_layout(); plt.savefig('fig2.png', dpi=200, bbox_inches='tight'); plt.show()
169
170 # =====
171 # 7. PART (b): PRICE DISCOVERY MEASURES
172 # =====
173
174 # --- (i) Gonzalo-Granger Component Share ---
175 denom_cs = alpha_F - alpha_S
176 CS_futures = -alpha_S / denom_cs
177 CS_spot = alpha_F / denom_cs
178
179 print(f"\n--- GONZALO-GRANGER COMPONENT SHARE ---")
180 print(f" CS_Futures = {CS_futures:.4f} ({CS_futures*100:.2f}%)")
181 print(f" CS_Spot = {CS_spot:.4f} ({CS_spot*100:.2f}%)")
182 if not (0 <= CS_futures <= 1):
183     print(" WARNING: Both alphas < 0 (same sign) -> formula outside [0,1];
      indeterminate.")
184
185 # --- (ii) Hasbrouck Information Share ---
186 MA_HORIZON = 3000
187 Psi1 = vecm_fit.ma_rep(maxn=MA_HORIZON).sum(axis=0)
188 psi = Psi1[0, :]
189 Omega = np.cov(vecm_fit.resid.T)
190 denom_IS = float(psi @ Omega @ psi)
191
192 # Ordering 1: Spot first (upper bound for futures)
193 F1 = np.linalg.cholesky(Omega)
194 psi_F1 = psi @ F1
195 IS_F_upper = float(psi_F1[1]**2 / denom_IS)
196 IS_S_lower = float(psi_F1[0]**2 / denom_IS)
197
198 # Ordering 2: Futures first (lower bound for futures)
199 Omega2 = Omega[[1,0],:][:,[1,0]]
200 F2 = np.linalg.cholesky(Omega2)
201 psi2 = psi[[1,0]]
202 psi_F2 = psi2 @ F2

```

```

206 IS_F_lower = float(psi_F2[0]**2 / denom_IS)
207 IS_S_upper = float(psi_F2[1]**2 / denom_IS)
208
209 IS_F_mid = (IS_F_upper + IS_F_lower) / 2
210 IS_S_mid = 1 - IS_F_mid
211
212 print(f"\n--- HASBROUCK INFORMATION SHARE ---")
213 print(f"   {'Market':<10} {'Lower':>8} {'Upper':>8} {'Midpoint':>10}")
214 print(f"   {'Futures':<10} {IS_F_lower:>8.4f} {IS_F_upper:>8.4f} {IS_F_mid:>10.4f}")
215 print(f"   {'Spot':<10} {IS_S_lower:>8.4f} {IS_S_upper:>8.4f} {IS_S_mid:>10.4f}")
216 print(f"   Note: 99.55% futures share is a scale artefact (~100x price-level
        difference).")
217
218 # --- (iii) Garbade-Silber Measure ---
219 spread_lag = (df['futures'] - df['spot']).shift(1)
220 gs_data = pd.DataFrame({'dS': df_diff['spot'],
221                        'dF': df_diff['futures'],
222                        'spread_lag': spread_lag}).dropna()
223 X_gs = add_constant(gs_data['spread_lag'])
224
225 ols_gs_s = OLS(gs_data['dS'], X_gs).fit()
226 a1 = float(ols_gs_s.params['spread_lag'])
227 p_a1 = ols_gs_s.pvalues['spread_lag']
228
229 ols_gs_f = OLS(gs_data['dF'], X_gs).fit()
230 b1 = float(ols_gs_f.params['spread_lag'])
231 p_b1 = ols_gs_f.pvalues['spread_lag']
232
233 GS_F = a1 / (a1 - b1)
234 GS_S = 1 - GS_F
235
236 print(f"\n--- GARBADE-SILBER MEASURE ---")
237 print(f"   Spot eq:   a1 = {a1:.6f}   (p={p_a1:.4f})   R2={ols_gs_s.rsquared:.4f}")
238 print(f"   Futures eq: b1 = {b1:.6f}   (p={p_b1:.4f})   R2={ols_gs_f.rsquared:.4f}")
239 print(f"   GS_F = {GS_F:.4f} ({GS_F*100:.2f}%) | GS_S = {GS_S:.4f}
        ({GS_S*100:.2f}%)")
240 print(f"   Note: Both coefficients insignificant; lagged spread has no predictive
        power.")
241
242 # --- Summary Table ---
243 avg_PD_F = np.mean([IS_F_mid, CS_futures, GS_F])
244 print(f"\n--- PRICE DISCOVERY SUMMARY ---")
245 print(f"   {'Measure':<28} {'Futures':>10} {'Spot':>10}")
246 print(f"   {'-'*50}")
247 print(f"   {'Hasbrouck IS (midpoint)':<28} {IS_F_mid*100:>9.2f}%
        {IS_S_mid*100:>9.2f}%")
248 print(f"   {'Gonzalo-Granger CS':<28} {CS_futures*100:>9.2f}% {CS_spot*100:>9.2f}%")
249 print(f"   {'Garbade-Silber':<28} {GS_F*100:>9.2f}% {GS_S*100:>9.2f}%")
250 print(f"   {'-'*50}")
251 print(f"   {'AVERAGE':<28} {avg_PD_F*100:>9.2f}% {(1-avg_PD_F)*100:>9.2f}%")
252 print(f"   All measures compromised by cross-exchange/currency mismatch.")
253
254 # Figure 3: Price Discovery Bar Chart
255 fig, ax = plt.subplots(figsize=(9, 5))
256 labels = ['Hasbrouck IS', 'Component Share', 'Garbade-Silber']
257 fut_vals = [IS_F_mid, CS_futures, GS_F]
258 spt_vals = [IS_S_mid, CS_spot, GS_S]
259 x, w = np.arange(3), 0.35

```

```

260 bars_f = ax.bar(x - w/2, [v*100 for v in fut_vals], w, label='Futures',
    color='steelblue')
261 bars_s = ax.bar(x + w/2, [v*100 for v in spt_vals], w, label='Spot',
    color='darkorange')
262 for bar in list(bars_f) + list(bars_s):
263     ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
264             f'{bar.get_height():.1f}%', ha='center', va='bottom', fontsize=10)
265 ax.axhline(50, color='red', ls='--', lw=0.9, label='50% line')
266 ax.set_xticks(x); ax.set_xticklabels(labels, fontsize=11)
267 ax.set_ylabel('Price Discovery Share (%)'); ax.set_ylim(0, 130)
268 ax.set_title('Price Discovery Shares      All Three Measures (Gold MCX/COMEX,
    2014  2024  )',
269               fontsize=11, fontweight='bold')
270 ax.legend(fontsize=10); ax.grid(axis='y', alpha=0.3)
271 plt.tight_layout(); plt.savefig('fig3.png', dpi=200, bbox_inches='tight'); plt.show()
272
273 # =====
274 # 8. PART (c): OPTIMAL HEDGE RATIO AND HEDGING EFFECTIVENESS
275 # =====
276 var_S = float(df_diff['spot'].var())
277 var_F = float(df_diff['futures'].var())
278 cov_SF = float(df_diff.cov().loc['spot', 'futures'])
279 corr_SF = float(df_diff.corr().loc['spot', 'futures'])
280
281 print(f"\n--- MOMENTS OF FIRST-DIFFERENCED SERIES ---")
282 print(f"  Var( S )      = {var_S:.2f}")
283 print(f"  Var( F )      = {var_F:.2f}")
284 print(f"  Cov( S , F )    = {cov_SF:.4f}")
285 print(f"  Corr( S , F )   = {corr_SF:.6f}")
286
287 def portfolio_var(h):
288     return var_S - 2*h*cov_SF + h**2 * var_F
289
290 # OLS hedge ratio: Delta_S = alpha + h* Delta_F + eta
291 X_ols = add_constant(df_diff['futures'])
292 ols_h = OLS(df_diff['spot'], X_ols).fit()
293 h_ols = float(ols_h.params['futures'])
294 HE_ols = float(ols_h.rsquared)
295
296 # VECM-adjusted: add lagged ECT as regressor
297 ect_lag = ect_series.shift(1)
298 vecm_h_data = pd.DataFrame({'dS': df_diff['spot'],
299                             'dF': df_diff['futures'],
300                             'ect_lag': ect_lag}).dropna()
301 X_vecm_h = add_constant(vecm_h_data[['dF', 'ect_lag']])
302 ols_vecm_h = OLS(vecm_h_data['dS'], X_vecm_h).fit()
303 h_vecm = float(ols_vecm_h.params['dF'])
304 phi_ecm = float(ols_vecm_h.params['ect_lag'])
305 HE_vecm = float(ols_vecm_h.rsquared)
306
307 var_ols_h = portfolio_var(h_ols)
308 var_vecm_h = portfolio_var(h_vecm)
309 var_naive = portfolio_var(1.0)
310
311 print(f"\n--- HEDGE RATIO RESULTS ---")
312 print(f"  h* (OLS)      = {h_ols:.6f}    HE = {HE_ols*100:.2f}%")
313 print(f"  h* (VECM)     = {h_vecm:.6f}    HE = {HE_vecm*100:.2f}%")
314 print(f"  phi (ECT)     = {phi_ecm:.6f}    p = {ols_vecm_h.pvalues['ect_lag']:.4f}")
315

```

```

316 print(f"\n--- VARIANCE REDUCTION TABLE ---")
317 print(f"   {'Strategy':<20} {'h*':>8} {'Port.Var':>14} {'Var.Red%':>12}")
318 print(f"   {'-'*57}")
319 for name, (h, vp) in [('Unhedged (h=0)', (0.0, var_S)),
320                      ('Naive (h=1)', (1.0, var_naive)),
321                      ('OLS Optimal', (h_ols, var_ols_h)),
322                      ('VECM-adjusted', (h_vecm, var_vecm_h))]:
323     red = (1 - vp / var_S) * 100
324     print(f"   {name:<20} {h:>8.4f} {vp:>14.2f} {red:>11.2f}%")
325
326 # Rolling hedge ratio (252-day window)
327 roll_h, roll_HE, roll_dates_list = [], [], []
328 for i in range(WINDOW, len(df_diff)):
329     w = df_diff.iloc[i - WINDOW : i]
330     X_r = add_constant(w['futures'])
331     m = OLS(w['spot'], X_r).fit()
332     roll_h.append(float(m.params['futures']))
333     roll_HE.append(float(m.rsquared))
334     roll_dates_list.append(df_diff.index[i])
335
336 roll_h = np.array(roll_h)
337 roll_HE = np.array(roll_HE)
338 roll_dates_arr = pd.DatetimeIndex(roll_dates_list)
339
340 print(f"\n--- ROLLING HEDGE RATIO (252-day) ---")
341 print(f"   Mean = {roll_h.mean():.4f} | Std = {roll_h.std():.4f}")
342 print(f"   Max = {roll_h.max():.4f} on {roll_dates_arr[np.argmax(roll_h)].date()}
343         (COVID rupee shock)")
344
345 # Figure 4: Rolling h* and HE
346 fig, axes = plt.subplots(2, 1, figsize=(13, 8), sharex=True)
347 axes[0].plot(roll_dates_arr, roll_h, color='forestgreen', lw=1.0, label='Rolling
348 $h^*$',)
349 axes[0].axhline(h_ols, color='black', lw=1.2, ls='--', label=f'OLS
350 $h^*${h_ols:.4f}')
351 axes[0].axhline(h_vecm, color='purple', lw=1.2, ls=':', label=f'VECM
352 $h^*${h_vecm:.4f}')
353 axes[0].axvspan(pd.Timestamp('2020-02-01'), pd.Timestamp('2020-08-31'),
354                 alpha=0.15, color='red', label='COVID-19')
355 axes[0].axvspan(pd.Timestamp('2022-02-24'), pd.Timestamp('2022-09-30'),
356                 alpha=0.15, color='orange', label='Russia-Ukraine')
357 axes[0].set_title('Panel A: Rolling 252-day Optimal Hedge Ratio', fontweight='bold',
358                 fontsize=11)
359 axes[0].set_ylabel('Hedge Ratio $h^*$',)
360 axes[0].legend(fontsize=9)
361 axes[0].grid(True, alpha=0.3)
362
363 axes[1].plot(roll_dates_arr, roll_HE * 100, color='steelblue', lw=1.0,
364             label='Rolling HE (%)')
365 axes[1].axhline(HE_ols * 100, color='black', lw=1.2, ls='--', label=f'OLS
366 HE={HE_ols*100:.2f}%')
367 axes[1].axvspan(pd.Timestamp('2020-02-01'), pd.Timestamp('2020-08-31'), alpha=0.15,
368                 color='red')
369 axes[1].axvspan(pd.Timestamp('2022-02-24'), pd.Timestamp('2022-09-30'), alpha=0.15,
370                 color='orange')
371 axes[1].set_title('Panel B: Rolling Hedging Effectiveness (%)', fontweight='bold',
372                 fontsize=11)
373 axes[1].set_ylabel('HE (%)')
374 axes[1].set_xlabel('Date')
375 axes[1].legend(fontsize=9)
376 axes[1].grid(True, alpha=0.3)
377 plt.tight_layout()
378 plt.savefig('fig4.png', dpi=200, bbox_inches='tight')
379 plt.show()

```

```

364
365 # Figure 5: Combined 3-panel summary
366 fig, axes = plt.subplots(3, 1, figsize=(13, 13))
367 axes[0].plot(df.index, df['spot'], label='Spot (INR/10g)', color='steelblue',
368             lw=0.9)
369 axes[0].plot(df.index, df['futures'], label='Futures (INR/10g)', color='darkorange',
370             lw=0.9, ls='--')
371 axes[0].set_title('Panel A: Gold Spot vs Back-Adjusted Continuous Futures',
372                 fontweight='bold', fontsize=11)
373 axes[0].set_ylabel('Price (INR/10g)'); axes[0].legend(fontsize=10);
374 axes[0].grid(True, alpha=0.3)
375
376 axes[1].plot(ect_series.index, ect_series, color='crimson', lw=0.7, label='ECT')
377 axes[1].axhline(ect_series.mean(), color='black', lw=0.9, ls='--', label='Mean ECT')
378 axes[1].fill_between(ect_series.index, ect_series, ect_series.mean(),
379                     where=(ect_series > ect_series.mean()), alpha=0.2, color='red', label='Spot
380                         relatively high')
381 axes[1].fill_between(ect_series.index, ect_series, ect_series.mean(),
382                     where=(ect_series < ect_series.mean()), alpha=0.2, color='navy', label='Futures
383                         relatively high')
384 axes[1].set_title('Panel B: Error Correction Term (ECT)', fontweight='bold',
385                 fontsize=11)
386 axes[1].set_ylabel('ECT Value'); axes[1].legend(fontsize=9); axes[1].grid(True,
387                 alpha=0.3)
388
389 axes[2].plot(roll_dates_arr, roll_h, color='forestgreen', lw=1.0, label='Rolling
390             $h^{*}$')
391 axes[2].axhline(h_ols, color='black', lw=1.2, ls='--', label=f'OLS
392             $h^{*}=\{h_{ols}:.4f\}$')
393 axes[2].axhline(h_vecm, color='purple', lw=1.2, ls=':', label=f'VECM
394             $h^{*}=\{h_{vecm}:.4f\}$')
395 axes[2].axvspan(pd.Timestamp('2020-02-01'), pd.Timestamp('2020-08-31'),
396                 alpha=0.15, color='red', label='COVID-19')
397 axes[2].axvspan(pd.Timestamp('2022-02-24'), pd.Timestamp('2022-09-30'),
398                 alpha=0.15, color='orange', label='Russia-Ukraine')
399 axes[2].set_title('Panel C: Rolling 252-day Optimal Hedge Ratio', fontweight='bold',
400                 fontsize=11)
401 axes[2].set_ylabel('Hedge Ratio $h^{*}$'); axes[2].set_xlabel('Date')
402 axes[2].legend(fontsize=9); axes[2].grid(True, alpha=0.3)
403 plt.tight_layout(); plt.savefig('fig5.png', dpi=200, bbox_inches='tight'); plt.show()
404
405 # =====
406 # 9. FINAL SUMMARY
407 # =====
408 print("\n" + "=" * 65)
409 print(" QUESTION 4 -- COMPLETE RESULTS SUMMARY")
410 print("=" * 65)
411 print(f"\n[PART a] VECM")
412 print(f" alpha_S = {alpha_S:.6f} -> Spot corrects {abs(alpha_S)*100:.2f}%/day;
413       half-life = {hl_S:.1f} days")
414 print(f" alpha_F = {alpha_F:.6f} -> Both markets share error correction")
415 print(f" ECT is I(0): ADF p = {p_ect:.6f}")
416
417 print(f"\n[PART b] PRICE DISCOVERY")
418 print(f" {'Measure':<28} {'Futures':>10} {'Spot':>10}")
419 print(f" {'-'*50}")
420 print(f" {'Hasbrouck IS (midpoint)':<28} {IS_F_mid*100:>9.2f}%
421       {IS_S_mid*100:>9.2f}% (scale artefact)")

```

```
409 print(f"    {'Gonzalo-Granger CS':<28} {CS_futures*100:>9.2f}% {CS_spot*100:>9.2f}%  
      (both    <0: indeterminate)")  
410 print(f"    {'Garbade-Silber':<28} {GS_F*100:>9.2f}% {GS_S*100:>9.2f}%    (both insig.)")  
411  
412 print(f"\n[PART c] HEDGE RATIO")  
413 print(f"    h*(OLS)   = {h_ols:.6f}    HE = {HE_ols*100:.2f}%    (low corr=0.085 limits  
      effectiveness)")  
414 print(f"    h*(VECM) = {h_vecm:.6f}    HE = {HE_vecm*100:.2f}%    (ECT adds ~10x  
      improvement)")  
415 print(f"    Naive h=1 increases variance by ~{abs((1-var_naive/var_S)*100):.0f}%  
      (scale mismatch catastrophic)")  
416 print(f"    Rolling h*: mean={roll_h.mean():.4f}, std={roll_h.std():.4f}; spike on  
      {roll_dates_arr[np.argmax(roll_h)].date()}")  
417 print(f"\n    Figures saved: fig1.png, fig2.png, fig3.png, fig4.png, fig5.png")  
418 print("=" * 65)
```

Code for Question 5

```
1  r = 0.06  
2  c = 0.03  
3  
4  df["F_theoretical"] = df["spot"] * np.exp((r + c) * df["T"])  
5  df["Spread"] = df["futures"] - df["F_theoretical"]  
6  
7  df.head()  
8  
9  #Identification of arbitrage opportunities  
10  
11  transaction_cost = 0.5  
12  
13  df["CashCarry"] = df["Spread"] > transaction_cost  
14  df["ReverseCashCarry"] = df["Spread"] < -transaction_cost  
15  
16  df=df.reset_index()  
17  df[["Date", "Spread", "CashCarry", "ReverseCashCarry"]].head()  
18  
19  # Plot for spread  
20  
21  plt.figure()  
22  plt.plot(df["Date"], df["Spread"])  
23  plt.axhline(0)  
24  plt.title("Futures Mispricing")  
25  plt.show()  
26  
27  import numpy as np  
28  
29  T1 = 10/365    # near  
30  T2 = 20/365    # mid  
31  T3 = 30/365    # far  
32  
33  df["F1"] = df["spot"] * np.exp((r+c)*T1)  
34  df["F2"] = df["spot"] * np.exp((r+c)*T2)  
35  df["F3"] = df["spot"] * np.exp((r+c)*T3)  
36  
37  # Butterfly spread  
38  df["Butterfly"] = df["F1"] - 2*df["F2"] + df["F3"]  
39
```

```
40
41 import matplotlib.pyplot as plt
42
43 plt.figure()
44 plt.plot(df["Date"], df["Butterfly"])
45 plt.axhline(0)
46 plt.title("Butterfly Spread")
47 plt.show()
```

Code for Question 6

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import requests
4
5 lat, lon = 26.4499, 80.3319
6
7 url = f"https://archive-api.open-meteo.com/v1/archive?latitude={lat}&longitude={lon}&
      start_date=2010-01-01&end_date=2020-12-31&daily=temperature_2m_mean&timezone=auto
      "
8
9 response = requests.get(url)
10 data = response.json()
11
12 df = pd.DataFrame({
13     "Date": pd.to_datetime(data["daily"]["time"]),
14     "Temp": pd.to_numeric(data["daily"]["temperature_2m_mean"], errors="coerce")
15 }).dropna()
16
17 df = df.sort_values("Date").set_index("Date")
18 df.head()
19
20 monthly_temp = df["Temp"].resample("M").mean()
21
22 # Plot monthly temperature series
23 plt.figure(figsize=(12, 5))
24 plt.plot(monthly_temp.index, monthly_temp.values, label="Monthly Avg Temp")
25 plt.title("Monthly Mean Temperature (2010 2020 )")
26 plt.xlabel("Date")
27 plt.ylabel("Temperature ( C )")
28 plt.grid(True)
29 plt.legend()
30 plt.tight_layout()
31 plt.show()
32
33 month_avg = monthly_temp.groupby(monthly_temp.index.month).mean()
34 plt.figure(figsize=(8, 4))
35 plt.plot(month_avg.index, month_avg.values, marker="o")
36 plt.title("Average Temperature by Calendar Month")
37 plt.xlabel("Month")
38 plt.ylabel("Temperature ( C )")
39 plt.xticks(range(1, 13))
40 plt.grid(True)
41 plt.tight_layout()
42 plt.show()
43
44
```

```
45 import requests
46 import pandas as pd
47 import numpy as np
48 import matplotlib.pyplot as plt
49 from statsmodels.tsa.statespace.sarimax import SARIMAX
50 from statsmodels.tsa.seasonal import seasonal_decompose
51 decomp = seasonal_decompose(monthly_temp, model="additive", period=12)
52
53 fig, axes = plt.subplots(4, 1, figsize=(12, 10), sharex=True)
54 axes[0].plot(decomp.observed)
55 axes[0].set_title("Observed")
56 axes[1].plot(decomp.trend)
57 axes[1].set_title("Trend")
58 axes[2].plot(decomp.seasonal)
59 axes[2].set_title("Seasonality")
60 axes[3].plot(decomp.resid)
61 axes[3].set_title("Residual")
62 plt.tight_layout()
63 plt.show()
64
65
66 base_temp = 18.0 # standard base temperature in C
67
68 df["HDD"] = np.maximum(base_temp - df["Temp"], 0)
69 df["CDD"] = np.maximum(df["Temp"] - base_temp, 0)
70
71 monthly_hdd = df["HDD"].resample("M").sum()
72 monthly_cdd = df["CDD"].resample("M").sum()
73
74 # Plot derivative index
75 plt.figure(figsize=(12, 5))
76 plt.plot(monthly_hdd.index, monthly_hdd.values, label="Monthly HDD")
77 plt.plot(monthly_cdd.index, monthly_cdd.values, label="Monthly CDD")
78 plt.title("Monthly Heating Degree Days (HDD) and Cooling Degree Days (CDD)")
79 plt.xlabel("Date")
80 plt.ylabel("Degree Days")
81 plt.grid(True)
82 plt.legend()
83 plt.tight_layout()
84 plt.show()
85
86 train = monthly_temp[:-12]
87 test = monthly_temp[-12:]
88
89 model = SARIMAX(
90     train,
91     order=(1, 1, 1),
92     seasonal_order=(1, 1, 1, 12),
93     enforce_stationarity=False,
94     enforce_invertibility=False
95 )
96
97 model_fit = model.fit(dispatch=False)
98 print(model_fit.summary())
99
100 forecast_test = model_fit.get_forecast(steps=len(test))
101 forecast_mean = forecast_test.predicted_mean
102 forecast_ci = forecast_test.conf_int()
103
```



```
104 # Plot actual vs forecast
105 plt.figure(figsize=(12, 5))
106 plt.plot(train.index, train.values, label="Training Data")
107 plt.plot(test.index, test.values, label="Actual Test Data")
108 plt.plot(forecast_mean.index, forecast_mean.values, label="Forecast")
109 plt.fill_between(
110     forecast_ci.index,
111     forecast_ci.iloc[:, 0],
112     forecast_ci.iloc[:, 1],
113     alpha=0.2
114 )
115 plt.title("SARIMA Forecast of Monthly Mean Temperature")
116 plt.xlabel("Date")
117 plt.ylabel("Temperature ( C )")
118 plt.grid(True)
119 plt.legend()
120 plt.tight_layout()
121 plt.show()
122
123 final_model = SARIMAX(
124     monthly_temp,
125     order=(1, 1, 1),
126     seasonal_order=(1, 1, 1, 12),
127     enforce_stationarity=False,
128     enforce_invertibility=False
129 ).fit(dispatch=False)
130
131 future_forecast = final_model.get_forecast(steps=12)
132 future_mean = future_forecast.predicted_mean
133 future_ci = future_forecast.conf_int()
134
135 plt.figure(figsize=(12, 5))
136 plt.plot(monthly_temp.index, monthly_temp.values, label="Historical Monthly Temp")
137 plt.plot(future_mean.index, future_mean.values, label="12-Month Forecast")
138 plt.fill_between(
139     future_ci.index,
140     future_ci.iloc[:, 0],
141     future_ci.iloc[:, 1],
142     alpha=0.2
143 )
144 plt.title("12-Month Forecast of Monthly Mean Temperature")
145 plt.xlabel("Date")
146 plt.ylabel("Temperature ( C )")
147 plt.grid(True)
148 plt.legend()
149 plt.tight_layout()
150 plt.show()
151
152
153 base_temp = 18
154
155 df["HDD"] = (base_temp - df["Temp"]).clip(lower=0)
156
157 plt.figure(figsize=(10,5))
158 plt.plot(df.index, df["HDD"])
159 plt.title("Heating Degree Days")
160 plt.show()
161
162
```

```
163 import numpy as np
164
165 # Simulate HDD payoff
166 strike = df["HDD"].mean()
167 simulated = np.linspace(strike*0.5, strike*1.5, 50)
168
169 payoff = simulated - strike
170
171 plt.plot(simulated, payoff)
172 plt.xlabel("HDD at Maturity")
173 plt.ylabel("Payoff")
174 plt.title("Payoff of HDD Futures Contract")
175 plt.show()
```

Code for Question 7

Use the code from Q3 to fetch data.

```
1 # log returns
2 df['r_spot'] = np.log(df['spot'] / df['spot'].shift(1))
3 df['r_fut'] = np.log(df['futures'] / df['futures'].shift(1))
4
5 df = df.dropna()
6
7 # remove extreme outliers
8 df['r_spot'] = df['r_spot'].clip(lower=df['r_spot'].quantile(0.001),
9     upper=df['r_spot'].quantile(0.999))
10 df['r_fut'] = df['r_fut'].clip(lower=df['r_fut'].quantile(0.001),
11     upper=df['r_fut'].quantile(0.999))
12
13 # 30-day rolling volatility
14 window = 30
15 df['vol_30'] = df['r_spot'].rolling(window).std() * np.sqrt(252)
16
17 plt.figure(figsize=(12,5))
18 plt.plot(df.index, df['vol_30'])
19 plt.title('30-Day Rolling Volatility of Spot Returns')
20 plt.ylabel('Volatility')
21 plt.xlabel('Date')
22 plt.grid(True)
23 plt.savefig('fig1_rolling_volatility.png', dpi=300, bbox_inches='tight')
24 plt.show()
25
26 \begin{figure}
27 \centering
28 \includegraphics[width=0.5\linewidth]{fig1_rolling_volatility.png}
29 \caption{Enter Caption}
30 \label{fig:placeholder}
31 \end{figure}
32
33 from arch import arch_model
34
35 # GARCH(1,1)
36 garch11 = arch_model(df['r_spot']*100, vol='GARCH', p=1, q=1, mean='Constant')
37 res_garch11 = garch11.fit()
38
39 print(res_garch11.summary())
40
41 cond_vol = res_garch11.conditional_volatility
```

```

38
39 plt.figure(figsize=(12,5))
40 plt.plot(df.index, cond_vol)
41 plt.title("GARCH Conditional Volatility")
42 plt.grid(True)
43
44 plt.savefig('fig2_garch_volatility.png', dpi=300, bbox_inches='tight')
45 plt.show()
46
47 alpha = res_garch11.params['alpha[1]']
48 beta = res_garch11.params['beta[1]']
49 print("alpha: ", alpha)
50 print("beta: ", beta)
51 print("Persistence (alpha + beta):", alpha + beta)
52
53 import statsmodels.api as sm
54 from arch import arch_model
55
56 # We scale it back down by dividing by 100. it acts as a proxy for epsilon t
57 df['resid_sq'] = (res_garch11.resid / 100)**2
58
59 # volatility of futures market recently
60 df['fut_activity'] = df['r_fut'].rolling(5).std()
61
62 # epsilon squared at time=t-1
63 df['resid_sq_lag'] = df['resid_sq'].shift(1)
64 df_var = df.dropna(subset=['resid_sq', 'resid_sq_lag', 'fut_activity'])
65
66 # 5. run regression with following variables
67 X = df_var[['resid_sq_lag', 'fut_activity']]
68 X = sm.add_constant(X) # adds a column of ones.
69 y = df_var['resid_sq']
70
71 model_var = sm.OLS(y, X).fit()
72 print("\n--- Part (c): Variance Equation Test (Two-Stage OLS) ---")
73 print(model_var.summary())
74
75 threshold = df['fut_activity'].median()
76
77 high = df[df['fut_activity'] > threshold]
78 low = df[df['fut_activity'] <= threshold]
79
80 # annualized volatility for both high and low
81 vol_high = high['r_spot'].std() * np.sqrt(252)
82 vol_low = low['r_spot'].std() * np.sqrt(252)
83
84 print("Volatility during HIGH futures activity:", vol_high)
85 print("Volatility during LOW futures activity :", vol_low)
86
87 # Plot
88 plt.figure(figsize=(7, 5))
89 bars = plt.bar(['Low Futures Activity', 'High Futures Activity'], [vol_low,
90 vol_high], color=['green', 'red'])
91 plt.title('Part (d): Spot Volatility vs Futures Activity Regimes')
92 plt.ylabel('Annualised Spot Volatility')
93
94 for bar in bars:
95     yval = bar.get_height()
96     plt.text(bar.get_x() + bar.get_width()/2, yval + 0.005, f"{yval:.4f}",

```

```
        ha='center', va='bottom')
96
97 plt.ylim(0, max(vol_low, vol_high) * 1.2)
98 plt.savefig('fig3_regime_comparison.png', dpi=300, bbox_inches='tight')
99 plt.show()
```